



DEEP LEARNING OPTIMIZATION

딥러닝 모델 학습을 위한 최적화 알고리즘

Optimization Algorithms for Deep Learning Model Fitting

Gradient Descent 부터 SGD, Momentum, RMSprop, Adam, AdamW 까지
Python / TensorFlow-Keras 옵티마이저 옵션 완전 정리

목차

Agenda

01 왜 최적화가 중요한가

손실 지형과 최적화의 도전 과제

02 1 차 방법 : GD / SGD

배치 · 확률적 경사하강법의 기초

03 모멘텀 기반 방법

Momentum, Nesterov Accelerated Gradient

04 적응적 학습률 방법 I

Adagrad, RMSprop, Adadelta

05 적응적 학습률 방법 II

Adam, AdamW, Nadam, Adamax

06 비교, 스케줄링, 실전 가이드

종합 비교표 · LR 스케줄링 · 선택 가이드

07 코드 실습 & 요약

Keras 코드 예제, 치트시트, 참고 문헌

학습 목표

Learning Objectives

1

핵심 원리 이해

경사하강법 계열 옵티마이저들이 공통적으로 따르는 파라미터 업데이트 프레임워크와, 각 방법이 이를 어떻게 개선하는지 이해한다.

2

수식 유도과 직관

Momentum, RMSprop, Adam 등 주요 옵티마이저의 업데이트 식을 유도하고, 물리적·통계적 직관 (관성, 이동평균, 바이어스 보정) 을 설명할 수 있다.

3

Keras API 활용

`tf.keras.optimizers` 의 각 클래스 (SGD, RMSprop, Adam, AdamW 등) 의 하이퍼파라미터를 이해하고 `model.compile()` 단계에서 올바르게 선택·설정한다.

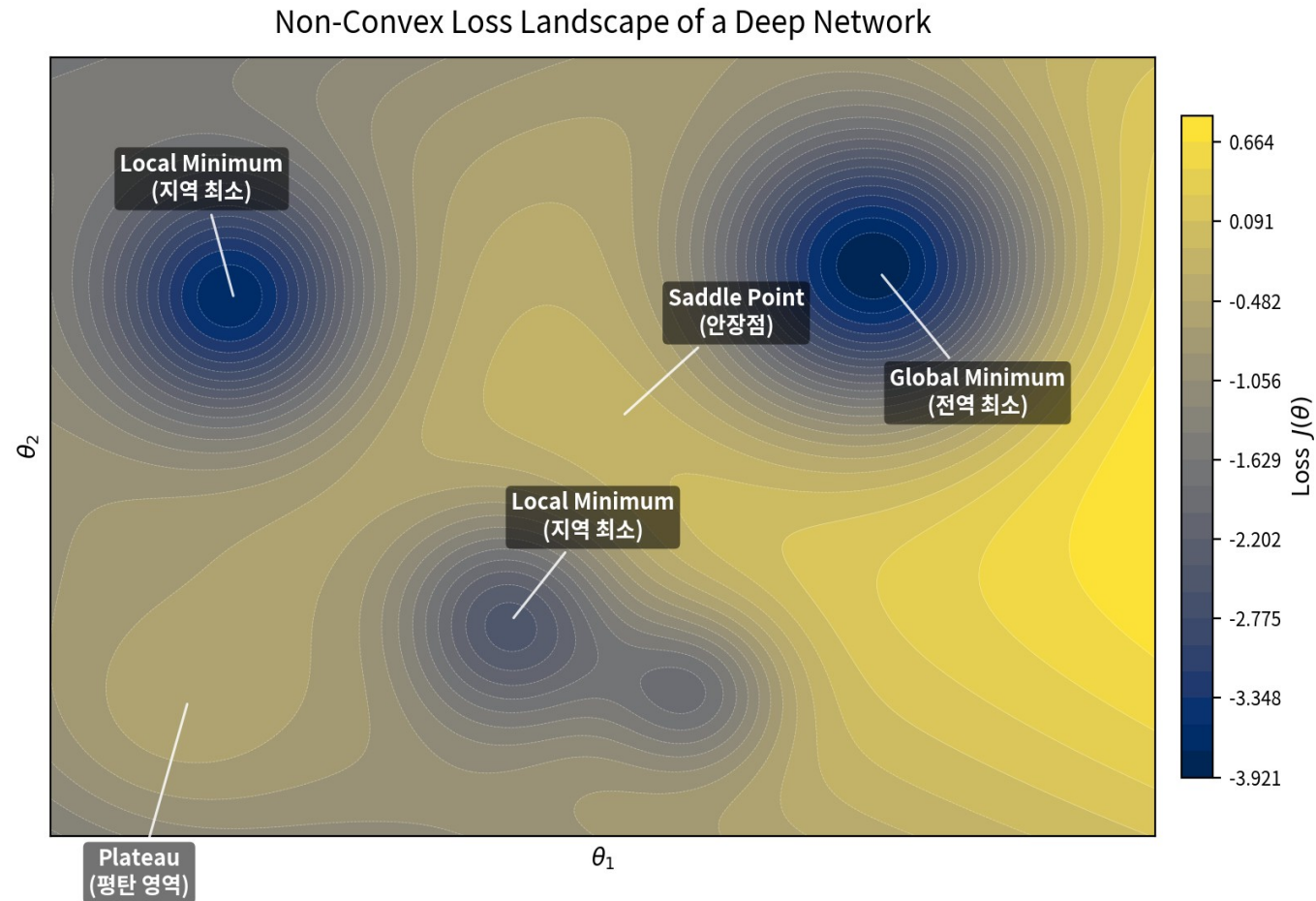
4

실전 선택 능력

모델 구조 (CNN·RNN·Transformer) 와 데이터 특성에 따라 적절한 옵티마이저와 학습률 스케줄을 선택하고, 흔한 실패 사례를 진단할 수 있다.

왜 최적화가 어려운가 : 손실 지형

The Non-Convex Loss Landscape



- 딥러닝의 손실함수 $J(\theta)$ 는 수백만 ~ 수십억 개의 파라미터에 대한 매우 비볼록 (non-convex) 함수이다 .
- 전역 최소점 (Global Minimum) 외에도 다수의 지역 최소점 (Local Minimum), 안장점 (Saddle Point), 평탄 영역 (Plateau) 이 존재한다 .
- 고차원 공간에서는 지역 최소보다 안장점이 훨씬 더 흔하며 , 이 지점에서 그래디언트가 0 에 가까워 학습이 정체될 수 있다 .
- 따라서 옵티마이저의 목표는 단순히 ' 내리막으로 이동 ' 하는 것을 넘어 , 이러한 지형을 효율적으로 통과하는 것이다 .

최적화 문제의 일반적 프레임워크

The General Update-Rule Framework

목표 : $\min_{\theta} J(\theta) = (1/N) \sum_i L(f(x_i; \theta), y_i)$

$g_t = \nabla_{\theta} J(\theta_{t-1})$ ← 그래디언트
계산

$u_t = \varphi(g_1, \dots, g_t)$ ← 업데이트량
계산 (옵티마이저별 상이)

$\theta_t = \theta_{t-1} - \eta \cdot u_t$ ← 파라미터
갱신

- 모든 경사 기반 옵티마이저는 이 3 단계 루프를 공유한다 :
그래디언트 계산 → 업데이트량 계산 → 파라미터 갱신.
- 옵티마이저 간의 차이는 오직 φ (과거 그래디언트들을 어떻게 가공하는가)에 있다.
- η (학습률)는 모든 방법에 공통적인, 가장 민감한 하이퍼파라미터이다.
- 이후 슬라이드에서는 SGD → Momentum → Adagrad → RMSprop → Adam 순으로 φ 가 어떻게 발전해왔는지 살펴본다.

배치 경사하강법 (Batch Gradient Descent)

Full-Batch Gradient Descent

$$g_t = (1/N) \sum_{i=1}^N \nabla_{\theta} L(f(x_i; \theta_{t-1}), y_i)$$
$$\theta_t = \theta_{t-1} - \eta \cdot g_t$$

- 전체 학습 데이터 N 개를 모두 사용해 한 번의 업데이트를 수행한다.
- 장점 : 그래디언트 추정이 정확하고 업데이트 방향이 안정적이다 (분산이 낮음).
- 단점 : 데이터셋이 크면 (N 이 매우 큼) 한 스텝의 비용이 매우 커지고 , 메모리에 전체 배치를 올려야 한다 .
- 딥러닝의 대규모 데이터셋에는 비현실적이므로 , 실무에서는 거의 사용하지 않는다 .

확률적 경사하강법과 미니배치

Stochastic & Mini-Batch Gradient Descent

미니배치 SGD (실무 표준)

$$g_t = (1/m) \sum_{i \in B_t} \nabla_{\theta} L(f(x_i; \theta_{t-1}), y_i)$$

$$\theta_t = \theta_{t-1} - \eta \cdot g_t \quad (m = \text{배치 크기}, m \ll N)$$

- Keras 에서 batch_size 인자가 바로 m 을 의미하며 , model.fit(x, y, batch_size=32) 처럼 지정한다 .
- m=1 이면 순수 SGD(온라인 학습) , m=N 이면 배치 GD 와 동일하다 . 실무에서는 보통 32~256 사이 값을 사용한다 .

배치 크기 m	장점	단점
작음 (예 : 1~32)	빠른 업데이트 , 정규화 효과 (노이즈)	그래디언트 추정 분산 큼 , 수렴 불안정
중간 (예 : 32~256)	속도 - 안정성 균형 , GPU 병렬화에 적합	여전히 튜닝이 필요한 하이퍼파라미터
큼 (예 : N 에 근접)	그래디언트 추정 정확 , 안정적 수렴	메모리 부담 大 , 일반화 성능 저하 가능성

Keras 에서 옵티마이저 지정하기 : 기본 패턴

Specifying an Optimizer in model.compile()

```
import tensorflow as tf
from tensorflow import keras
model = keras.Sequential([...])

# 방법 1) 문자열 - 기본 하이퍼파라미터 사용
model.compile(optimizer="adam",
               loss="sparse_categorical_crossentropy")
# 방법 2) 인스턴스 - 하이퍼파라미터 직접 제어 (권장)
optimizer = keras.optimizers.Adam(
    learning_rate=1e-3, beta_1=0.9,
    beta_2=0.999, epsilon=1e-7)
model.compile(optimizer=optimizer, loss="...",
               metrics=["accuracy"])
history = model.fit(x_train, y_train, batch_size=32,
                    epochs=20, validation_data=(x_val, y_val))
```

`keras.optimizers` 모듈은 `tf.keras.optimizers` 와 동일 — TensorFlow 2.x 기준

문자열 vs 인스턴스

문자열 ("adam") 은 빠른 프로토타이핑에 유용하지만 기본값만 사용됩니다.

학습률 (learning_rate) 등 하이퍼파라미터를 조정하려면 반드시 옵티마이저 클래스의 인스턴스를 생성해서 전달해야 합니다.

이후 슬라이드들에서 각 옵티마이저 클래스의 실제 생성자 인자와 기본값을 다룹니다.

SGD 가 겪는 대표적인 어려움

Challenges of Vanilla (Mini-Batch) SGD

①

병목 지형 (Ill-Conditioning)

손실 표면의 곡률이 방향마다 크게 다르면 (가늘고 긴 협곡 형태), 고정 학습률로는 가파른 방향에서 진동하고 완만한 방향에서는 매우 느리게 진행된다 .

②

안장점 & 평탄 영역

고차원 파라미터 공간에서는 지역 최소보다 안장점이 압도적으로 많다 .
그래디언트가 0 에 가까워지면 SGD 는 정체되거나 매우 느리게 탈출한다 .

③

노이즈가 많은 그래디언트

미니배치로 추정한 그래디언트는 전체 데이터의 참 그래디언트에 대한 잡음 섞인 추정치이므로 , 매 스텝의 방향이 흔들린다 .

④

학습률 선택의 어려움

너무 크면 발산하거나 진동하고 , 너무 작으면 학습이 지나치게 느리다 .
게다가 모든 파라미터에 동일한 학습률을 적용하는 것이 최적일 아닐 수 있다 .

모멘텀 (SGD with Momentum)

Accelerating SGD with a Velocity Term

$$v_t = \beta \cdot v_{t-1} + (1-\beta) \cdot g_t$$

$$\theta_t = \theta_{t-1} - \eta \cdot v_t$$

$$(\beta \approx 0.9, \quad v_0 = 0)$$

- 직관 : 언덕을 굴러 내려가는 공처럼 , 과거 그래디언트 방향의 ' 관성 (velocity)' 을 누적한다 .
- 일관된 방향 (완만한 방향) 으로는 속도가 누적되어 가속되고 , 방향이 자주 바뀌는 축 (가파른 방향) 에서는 서로 상쇄되어 진동이 줄어든다 .
- v_t 는 그래디언트의 지수이동평균 (EMA) 으로 해석할 수 있다 — 이후 RMSprop·Adam 에서도 동일한 EMA 아이디어가 반복된다 .
- Keras: `keras.optimizers.SGD(learning_rate=0.01, momentum=0.9)`

네스테로프 가속 그래디언트 (NAG)

Nesterov Accelerated Gradient

$$\tilde{\theta}_t = \theta_{t-1} - \eta \cdot \beta \cdot v_{t-1} \quad \leftarrow$$

lookahead 지점

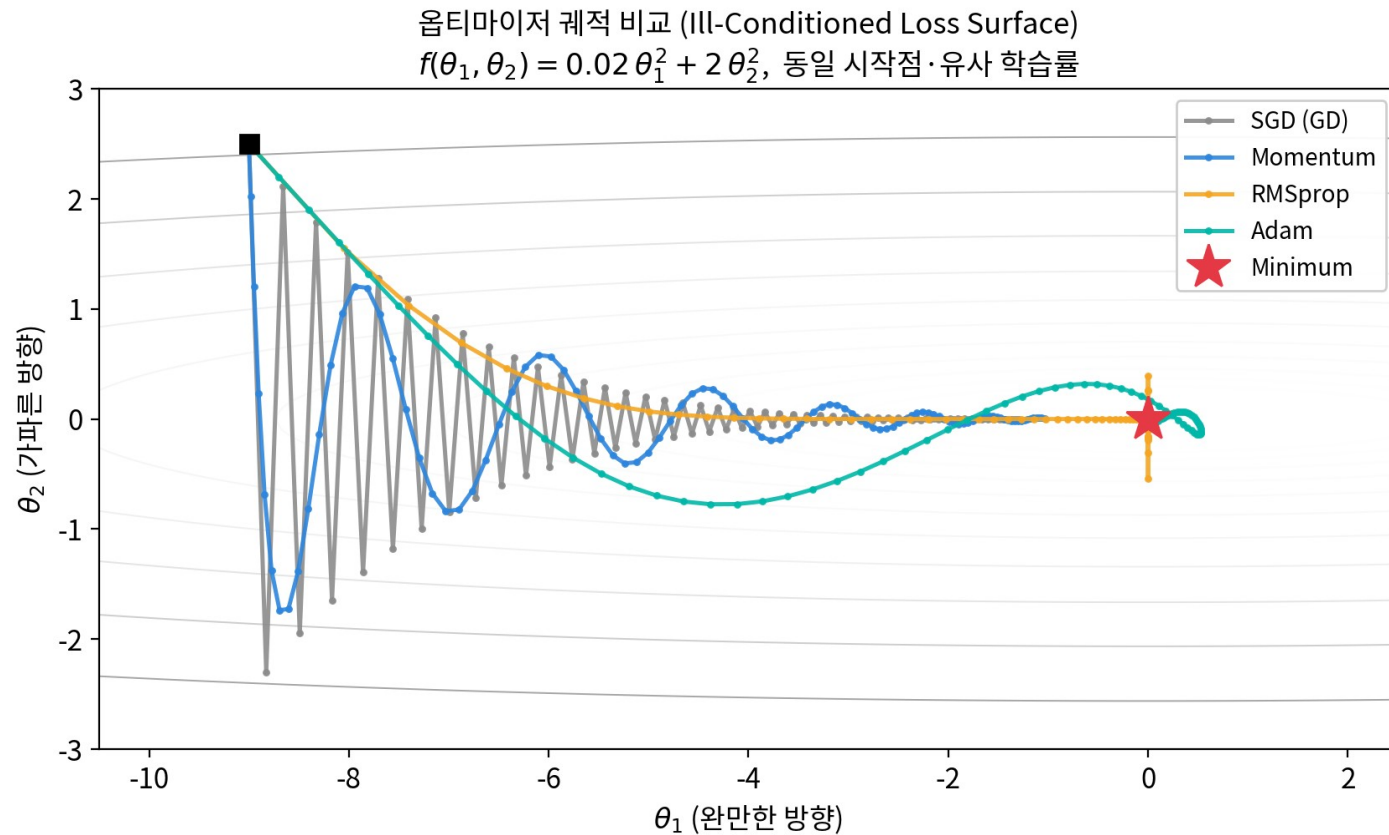
$$v_t = \beta \cdot v_{t-1} + (1-\beta) \cdot \nabla J(\tilde{\theta}_t)$$

$$\theta_t = \theta_{t-1} - \eta \cdot v_t$$

- 직관 : ' 일단 관성 방향으로 미리 가본 뒤 (lookahead), 그 지점에서 그래디언트를 측정 ' 한다 — 앞을 내다보고 브레이크를 미리 거는 효과 .
- 일반 Momentum 보다 최소점 근처에서의 오버슈팅 (overshoot) 과 진동이 적어 , 더 빠르고 안정적으로 수렴하는 경향이 있다 .
- Sutskever et al. (2013) 이 딥러닝 맥락에서 재조명하며 널리 사용되기 시작했다 .
- Keras: `keras.optimizers.SGD(learning_rate=0.01, momentum=0.9, nesterov=True)`

궤적 시각화 : 병목 지형에서의 옵티마이저

Trajectory Comparison on an Ill-Conditioned Surface

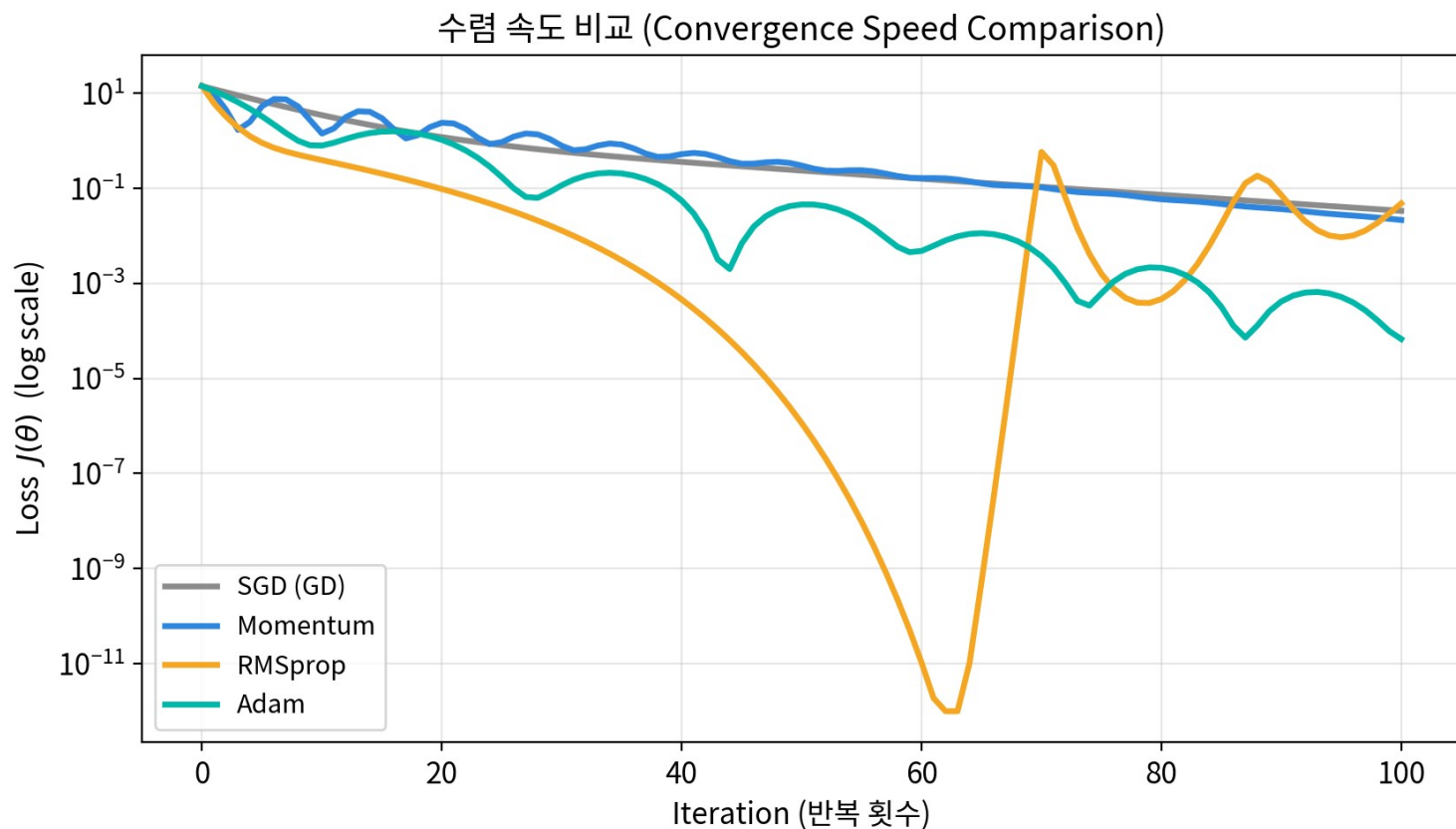


- 동일한 시작점에서 출발해도 옵티마이저별 경로가 크게 다르다 .
- SGD(회색) 는 가파른 축에서 심하게 지그재그 (진동) 하며 완만한 축으로는 매우 느리게 이동한다 .
- Momentum(파랑) 은 진동을 어느 정도 줄이지만 여전히 관성으로 인해 오버슈팅이 남아있다 .
- RMSprop·Adam(주황 · 초록) 은 축마다 학습률을 다르게 적용해 훨씬 직선에 가깝게 최소점에 도달한다 .

numpy 로 직접 구현한 각 옵티마이저 업데이트 식을 동일 손실함수 $f(\theta)=0.02\theta_1^2+2\theta_2^2$ 에 적용한 결과

수렴 속도 비교

Convergence Speed Comparison (Log Scale)



- γ 축이 로그 스케일임에 유의 — 적응형 방법 (RMSprop, Adam) 의 손실 감소 속도가 SGD·Momentum 보다 수십 ~ 수백 배 빠르다 .
- 적응형 방법도 최소점 근처에서는 진동이 완전히 사라지지 않을 수 있다 — 분모의 $\sqrt{\hat{v} + \epsilon}$ 항이 0 으로 수렴하지 않기 때문이다 .
- 실무 시사점 : 학습 초반 손실이 정체된다면 무조건 더 오래 기다리기보다 옵티마이저 · 학습률 선택을 재검토해야 한다 .

Keras 코드 : SGD 계열 옵티마이저

keras.optimizers.SGD — Options in Practice

```
# 1) 순수 SGD
opt1 = keras.optimizers.SGD(learning_rate=0.01)

# 2) SGD + Momentum ( 고전적 관성 방법 )
opt2 = keras.optimizers.SGD(learning_rate=0.01,
                             momentum=0.9)

# 3) SGD + Nesterov Momentum
opt3 = keras.optimizers.SGD(learning_rate=0.01,
                             momentum=0.9,
                             nesterov=True)

model.compile(optimizer=opt2, loss="categorical_crossentropy")
```

SGD 기본값

```
learning_rate = 0.01
momentum = 0.0
nesterov = False
weight_decay = None
```

momentum=0 이면 opt1 과 동일한
순수 SGD 가 됩니다 .

CNN 이미지 분류 등에서는
SGD+momentum 이 Adam 류보다 더
나은 일반화 성능을 보이는 경우가
많습니다 (17 번 참고).

출처: Keras 공식 문서 keras.io/api/optimizers/sgd (2026 년 기준)

왜 적응적 학습률이 필요한가 ?

Motivation for Per-Parameter Adaptive Learning Rates

- 지금까지의 방법 (SGD, Momentum, NAG) 은 모든 파라미터에 동일한 학습률 η 를 적용한다 .
- 그러나 실제 딥러닝 파라미터들은 매우 다른 통계적 성질을 가진다 :
 - 희소 특성 (sparse features): 단어 임베딩처럼 특정 파라미터는 아주 가끔만 업데이트된다 — 이런 파라미터는 업데이트될 때 더 큰 스텝이 유리하다 .
 - 곡률 차이 : 어떤 파라미터 방향은 손실이 급격히 변하고 (가파름), 어떤 방향은 완만하다 .
 - 학습 단계별 변화 : 학습 초반과 후반에 필요한 유효 학습률이 다를 수 있다 .
- 해결 아이디어 : 각 파라미터마다 ' 자기만의 ' 학습률을 그래디언트의 과거 이력에 기반해 자동으로 조절하자 .
- 이 아이디어의 계보 : Adagrad (2011) → RMSprop / Adadelta (2012) → Adam (2015) → AdamW (2019)

Adagrad (Adaptive Gradient Algorithm)

Duchi, Hazan & Singer (2011)

$$G_t = G_{t-1} + g_t \odot g_t \quad (\text{좌표별 그래디언트 제곱 누적})$$

$$\theta_t = \theta_{t-1} - \eta / (\sqrt{G_t} + \epsilon) \odot g_t$$

- 각 파라미터의 학습률을 '지금까지 누적된 그래디언트 크기'에 반비례하도록 조정 — 자주 / 크게 업데이트된 파라미터는 학습률이 작아진다.
- 장점: 희소 데이터 (NLP 임베딩 등)에 특히 강함 — 드물게 등장하는 특성에 상대적으로 큰 스텝을 준다.
- 치명적 단점: G_t 가 단조 증가만 하므로 유효 학습률이 계속 감소해 결국 학습이 사실상 멈춘다.
- Keras: `keras.optimizers.Adagrad(learning_rate=0.001, initial_accumulator_value=0.1, epsilon=1e-7)`

Adagrad 의 한계를 해결하기 : 지수이동평균

From Cumulative Sum to Exponential Moving Average

핵심 아이디어 : ' 모든 과거 ' 를 동일 가중치로 누적하는 대신 , 최근 그래디언트에 더 큰 가중치를 주는 지수이동평균 (EMA) 을 사용한다 .

$$E_t = \rho \cdot E_{t-1} + (1-\rho) \cdot g_t^2 \quad (\rho \approx 0.9 \sim 0.95, \text{ 오래된 정보는 점차 잊혀짐 })$$



RMSprop

Hinton 의 Coursera 강의 (2012, 비공식) 에서 제안 . E_t 만으로 학습률을 정규화 — 구현이 단순하고 RNN 학습에 특히 강함 .



Adadelta

Zeiler (2012). 학습률 η 자체를 없애고 , 과거 ' 파라미터 업데이트량 ' 의 이동평균으로 스텝 크기를 결정 — 스케일 불변성 확보 .

RMSprop (Root Mean Square Propagation)

Tieleman & Hinton, unpublished (2012)

$$E_t = \rho \cdot E_{t-1} + (1-\rho) \cdot g_t^2$$

$$\theta_t = \theta_{t-1} - \eta / (\sqrt{E_t} + \epsilon) \cdot g_t$$

($\rho = 0.9$, $\epsilon = 1e-7$ — Keras
기본값)

- Adagrad 의 ' 학습률이 0 으로 수렴 ' 하는 문제를 EMA 로 해결 — 최근 그래디언트 크기에 따라 학습률이 계속 적응한다 .
- RNN/LSTM 처럼 그래디언트 스케일이 시점마다 크게 변하는 모델 학습에 특히 안정적이어서 널리 사용되어 왔다 .
- Keras 구현은 momentum 옵션 (기본 0.0) 도 함께 제공해 RMSprop+Momentum 조합도 가능하다 .
- Keras: `keras.optimizers.RMSprop(learning_rate=0.001, rho=0.9, momentum=0.0, epsilon=1e-7, centered=False)`

Adadelta

Zeiler (2012) — Learning-Rate-Free Variant

$$E_t = \rho \cdot E_{t-1} + (1-\rho) \cdot g_t^2$$

$$\Delta\theta_t = - \left(\sqrt{(D_{t-1} + \epsilon)} / \sqrt{(E_t + \epsilon)} \right) \cdot$$

g_t

$$D_t = \rho \cdot D_{t-1} + (1-\rho) \cdot \Delta\theta_t^2$$

$$\theta_t = \theta_{t-1} + \Delta\theta_t$$

- RMSprop 과 유사하게 E_t (그래디언트 제곱 EMA) 를 사용하지만 , 분자도 ' 과거 업데이트량의 EMA' D_t 로 대체해 전역 학습률 η 를 제거한다 .
- 장점 : 단위 (unit) 가 일치하는 업데이트를 만들어 학습률을 따로 튜닝할 필요가 크게 줄어든다 .
- 실무에서는 RMSprop·Adam 만큼 널리 쓰이지는 않지만 , 하이퍼파라미터 탐색 자원이 부족할 때 유용한 대안이다 .
- Keras: `keras.optimizers.Adadelta(learning_rate=0.001, rho=0.95, epsilon=1e-7)` — 프레임워크 편의상 `learning_rate` 인자가 남아있음

Keras 코드 & 기본값 : Adagrad · RMSprop · Adadelata

Constructor Defaults (Keras / TensorFlow, 2026 기준)

옵티마이저	생성자 코드	주요 기본값
Adagrad	<code>keras.optimizers.Adagrad(learning_rate=0.001)</code>	<code>initial_accumulator_value=0.1 epsilon=1e-7</code>
RMSprop	<code>keras.optimizers.RMSprop(learning_rate=0.001)</code>	<code>rho=0.9, momentum=0.0 epsilon=1e-7, centered=False</code>
Adadelata	<code>keras.optimizers.Adadelata(learning_rate=0.001)</code>	<code>rho=0.95 epsilon=1e-7</code>

공통 인자 (모든 옵티마이저): `weight_decay, clipnorm, clipvalue, global_clipnorm, use_ema, ema_momentum` 등 — 27 번 슬라이드 참고

Adam (Adaptive Moment Estimation)

Kingma & Ba (2015) — the De-Facto Default

$$m_t = \beta_1 \cdot m_{t-1} + (1-\beta_1) \cdot g_t \quad \leftarrow$$

1 차 모멘트 (Momentum)

$$v_t = \beta_2 \cdot v_{t-1} + (1-\beta_2) \cdot g_t^2 \quad \leftarrow$$

2 차 모멘트 (RMSprop)

$$\hat{m}_t = m_t / (1-\beta_1^t), \quad \hat{v}_t =$$

$$v_t / (1-\beta_2^t) \quad \leftarrow \text{바이어스 보정}$$

$$\theta_t = \theta_{t-1} - \eta \cdot \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon)$$

- Adam = Momentum(방향의 관성, m_t) + RMSprop(파라미터별 학습률 정규화, v_t) 의 결합 .
- 그래디언트의 1 차 모멘트 (평균) 와 2 차 모멘트 (비중심 분산) 를 모두 추정해 업데이트에 사용한다 .
- 바이어스 보정 (다음 슬라이드) 이 있어 학습 초기부터 안정적인 추정치를 제공한다 .
- Keras 기본값 : learning_rate=0.001, beta_1=0.9, beta_2=0.999, epsilon=1e-7, amsgrad=False

Adam 의 바이어스 보정은 왜 필요한가 ?

Why Bias Correction Matters

$m_0 = 0, \quad v_0 = 0$ 으로 초기화

$$m_1 = \beta_1 \cdot 0 + (1 - \beta_1) \cdot g_1 = (1 - \beta_1) \cdot g_1$$

→ $\beta_1=0.9$ 이면 $m_1 = 0.1 \cdot g_1$ (참값의 10%로 과소평가!)

보정 : $\hat{m}_1 = m_1 / (1 - \beta_1^1) = m_1 / 0.1 = g_1 \quad \checkmark$

→ 보정 후에는 편향 없이 g_1 그대로 복원된다 .

- 이동평균을 0으로 초기화하면 학습 초반 몇 스텝 동안 추정치가 실제보다 훨씬 작게 (0 쪽으로 편향되어) 나온다 .
- $1/(1-\beta^t)$ 보정항은 t 가 작을 때 크게 확대 보정하고 , t 가 커질수록 ($\beta^t \rightarrow 0$) 보정항이 1에 수렴해 자연스럽게 사라진다 .
- 특히 $\beta_2=0.999$ 처럼 1에 가까운 값을 쓸 때는 v 의 초기 편향이 매우 크므로 , 이 보정이 없으면 학습 초반 학습률이 비정상적으로 커질 수 있다 .

AdamW — 분리된 가중치 감쇠

Loshchilov & Hutter (2019), Decoupled Weight Decay

[일반 Adam + L2 정규화] $g_t \leftarrow g_t + \lambda \cdot \theta_{t-1}$ (그래디언트에 섞임 $\rightarrow v$ 에도 영향)

[AdamW] $\theta_t = \theta_{t-1} - \eta \cdot (\hat{m}_t / (\sqrt{\hat{v}_t + \epsilon}) + \lambda \cdot \theta_{t-1})$ 적응적 스텝 가중치 감쇠 (독립적으로 적용)

- 문제 : Adam 에서 L2 정규화를 단순히 손실에 더하면 , 정규화 항까지 v_t (2 차 모멘트) 로 나뉘어 스케일이 왜곡된다 — 원래 의도한 만큼 감쇠되지 않음 .
- 해결 : 가중치 감쇠항 $\lambda \cdot \theta$ 를 그래디언트 적응 스텝과 분리하여 파라미터 업데이트에 직접 별도로 적용한다 .
- Transformer 계열 (BERT, GPT, ViT 등) 학습에서 AdamW + 코사인 스케줄이 사실상 표준 조합이다 .
- Keras 기본값 : learning_rate=0.001, weight_decay=0.004, beta_1=0.9, beta_2=0.999, epsilon=1e-7

Nadam 과 Adamax

Two Notable Adam Variants

Nadam

Dozat (2016) — Nesterov + Adam

$$\theta_t = \theta_{t-1} - \eta / (\sqrt{\hat{v}_t + \epsilon}) \cdot (\beta_1 \cdot \hat{m}_t + (1 - \beta_1) g_t / (1 - \beta_1^t))$$

- Adam 의 모멘텀 항 m_t 대신, NAG 처럼 '한 스텝 미리 내다본' lookahead 모멘텀을 사용한다.
- Adam 보다 수렴 초반이 약간 더 빠른 경우가 보고되며, RNN 계열에서 종종 선호된다.

Keras: `keras.optimizers.Nadam(learning_rate=0.001, beta_1=0.9, beta_2=0.999, epsilon=1e-7)`

Adamax

Kingma & Ba (2015) — L_∞ 노름 기반

$$u_t = \max(\beta_2 \cdot u_{t-1}, |g_t|)$$

$$\theta_t = \theta_{t-1} - (\eta / (1 - \beta_1^t)) \cdot m_t / u_t$$

- 2 차 모멘트에 L_2 노름 (제곱평균) 대신 L_∞ 노름 (최댓값) 을 사용 — 극단적으로 큰 그래디언트에 덜 민감하다.
- 임베딩 레이어처럼 그래디언트가 매우 희소하고 값의 편차가 큰 경우에 종종 더 안정적이다.

Keras: `keras.optimizers.Adamax(learning_rate=0.001, beta_1=0.9, beta_2=0.999, epsilon=1e-7)`

Keras 코드 & 기본값 : Adam 계열

Adam / AdamW / Nadam / Adamax — Constructor Defaults

옵티마이저	생성자 코드	주요 기본값
Adam	<code>keras.optimizers.Adam(learning_rate=0.001)</code>	<code>beta_1=0.9, beta_2=0.999 epsilon=1e-7, amsgrad=False</code>
AdamW	<code>keras.optimizers.AdamW(learning_rate=0.001)</code>	<code>weight_decay=0.004 beta_1=0.9, beta_2=0.999</code>
Nadam	<code>keras.optimizers.Nadam(learning_rate=0.001)</code>	<code>beta_1=0.9, beta_2=0.999 epsilon=1e-7</code>
Adamax	<code>keras.optimizers.Adamax(learning_rate=0.001)</code>	<code>beta_1=0.9, beta_2=0.999 epsilon=1e-7</code>

출처: Keras 공식 문서 keras.io/api/optimizers (2026 년 기준). `amsgrad=True` 로 설정하면 4 개 모두 AMSGrad 변형 적용 가능

옵티마이저 종합 비교표

Master Comparison of All Covered Optimizers

옵티마이저	핵심 메커니즘	파라미터당 추가 메모리	대표 학습률	강점
SGD	고정 학습률	0	0.01~0.1	단순, 이론적으로 잘 이해됨
Momentum	그래디언트 EMA(1 차)	1×	0.01~0.1	협곡 지형에서 가속
NAG	lookahead 모멘텀	1×	0.01~0.1	오버슈팅 감소
Adagrad	누적 제곱합	1×	0.01	희소 특성 (NLP 임베딩)
RMSprop	제곱 그래디언트 EMA	1×	0.001	RNN, 비정상 목적함수
Adadelta	업데이트량 EMA	2×	(η 불필요)	학습률 튜닝 최소화
Adam	1 차 + 2 차 모멘트 + 보정	2×	0.001~0.0003	범용 기본 선택
AdamW	Adam + 분리된 WD	2×	0.0003~0.001	Transformer / 대규모 모델
Nadam	Adam + NAG lookahead	2×	0.001~0.002	빠른 초기 수렴

‘파라미터당 추가 메모리’는 원본 파라미터 외에 저장해야 하는 상태 텐서 개수 (예: Adam 은 m, v 두 개 $\rightarrow 2\times$)

학습률 스케줄링과 Warmup

Learning Rate Schedules Complement the Optimizer

```
# 1) 지수 감쇠 (Exponential Decay)
lr_schedule = keras.optimizers.schedules.ExponentialDecay(
    initial_learning_rate=1e-3, decay_steps=10000, decay_rate=0.9)

# 2) 코사인 감쇠 + Warmup (Transformer 계열 표준)
lr_schedule = keras.optimizers.schedules.CosineDecay(
    initial_learning_rate=1e-4, decay_steps=100000,
    warmup_target=1e-3, warmup_steps=4000)

optimizer = keras.optimizers.AdamW(learning_rate=lr_schedule,
                                    weight_decay=1e-4)

# 3) 정체 시 자동 감소 (콜백 방식)
callback = keras.callbacks.ReduceLROnPlateau(
    monitor="val_loss", factor=0.5, patience=5, min_lr=1e-6)
model.fit(x, y, callbacks=[callback])
```

learning_rate 예 float 대신 스케줄 객체를 전달하면 옵티마이저가 매 스텝 자동으로 조회합니다

왜 필요한가 ?

옵티마이저 선택만큼이나 학습률의 '시간에 따른 변화'가 최종 성능에 큰 영향을 준다.

Warmup: 학습 초반 큰 학습률은 불안정
→ 작게 시작해 서서히 목표치까지 올림.

Decay: 학습 후반 학습률을 낮춰 손실
표면의 좁은 최소점 근처에 정밀하게 수렴.

Adam 계열도 스케줄과 함께 쓰는 것이
사실상 표준입니다.

실전 가이드 : 어떤 옵티마이저를 선택할까 ?

Practical Guidelines by Model / Task

CNN

이미지 분류 (CNN, ResNet 등)

SGD + Momentum(0.9) + 학습률 스케줄 (step/cosine decay) 이 최종 일반화 성능에서 Adam 류를 능가하는 경우가 많음 . 학습 시간 여유가 있을 때 권장 .

RNN

RNN / LSTM 시퀀스 모델

RMSprop 또는 Adam. 그래디언트 스케일 변화가 큰 시퀀스 데이터에서 적응적 학습률이 안정성을 크게 높여줌 .

TF

Transformer / LLM 계열

AdamW + Warmup + Cosine(또는 Linear) Decay 가 사실상 표준 .
weight_decay 는 보통 0.01~0.1 사이에서 튜닝 .

GAN

GAN, 소규모 데이터 / 빠른 프로토타이핑

GAN 은 Adam(beta_1=0.5) 이 관행적으로 사용됨 . 빠른 실험 단계에서는 기본값 Adam(lr=1e-3~1e-4) 으로 우선 베이스라인 확보 .

흔한 실수와 튜닝 팁

Common Pitfalls & Tuning Tips

- 학습률이 너무 크다 : 손실이 발산하거나 NaN 이 발생 → 학습률을 10 배 낮추거나 clipnorm/clipvalue 로 그래디언트 클리핑 적용 .
 - `keras.optimizers.Adam(learning_rate=1e-3, clipnorm=1.0)`
- 학습률이 너무 작다 : 손실이 거의 줄지 않음 → learning rate range test 로 적정 구간을 먼저 탐색 .
- Adam + L2 정규화 (kernel_regularizer) 를 함께 쓰면 의도한 만큼 정규화되지 않을 수 있음 → AdamW 의 weight_decay 사용을 고려 .
- β_1, β_2 는 거의 튜닝하지 않는다 : 기본값 (0.9, 0.999) 이 대부분의 상황에서 잘 작동 — 우선순위는 항상 학습률 (η) > 배치 크기 > β 값 .
- 혼합정밀도 (mixed precision) 학습 시 매우 작은 그래디언트가 0 으로 언더플로우할 수 있음 → loss_scale_factor 등 loss scaling 필요 .
- 검증 손실이 흔들린다면 배치 크기를 늘리거나 학습률 스케줄 (ReduceLROnPlateau, CosineDecay) 을 함께 사용 .

전체 코드 예제 : 옵티마이저 비교 실험

```
def build_model():
    return keras.Sequential([
        keras.layers.Dense(128, activation="relu", input_shape=(784,)),
        keras.layers.Dense(64, activation="relu"),
        keras.layers.Dense(10, activation="softmax"),
    ])

optimizers = {
    "SGD+Momentum": keras.optimizers.SGD(learning_rate=0.01, momentum=0.9),
    "RMSprop":      keras.optimizers.RMSprop(learning_rate=0.001),
    "Adam":         keras.optimizers.Adam(learning_rate=0.001),
    "AdamW":        keras.optimizers.AdamW(learning_rate=0.001, weight_decay=1e-4),
}

for name, opt in optimizers.items():
    model = build_model()
    model.compile(optimizer=opt, loss="sparse_categorical_crossentropy",
                  metrics=["accuracy"])
    h = model.fit(x_train, y_train, validation_split=0.1,
                  epochs=15, batch_size=128, verbose=0)
    print(f"{name}: val_acc={h.history['val_accuracy'][-1]:.4f}")
```

동일 모델· 동일 데이터에서 옵티마이저만 바꿔가며 val_loss 를 비교 시각화하는 것을 과제로 권장 (End-to-End: Comparing Optimizers on the Same Model)

핵심 요약 치트시트

One-Slide Cheat Sheet

확신이 없다면 ?

Adam(lr=1e-3) 또는 AdamW 로 시작 — 대부분의 문제에서 견고한 베이스라인 .

최고의 일반화가 필요하다면 ?

SGD+Momentum(0.9) + 학습률 스케줄 , 학습 시간을 더 투자 .

Transformer / LLM 이라면 ?

AdamW + Warmup + Cosine Decay + weight_decay 튜닝 .

희소 입력 (NLP 임베딩) 이라면 ?

Adagrad 또는 Adam — 드문 특성에 큰 스텝 적용 .

기억할 한 문장 : 옵티마이저는 “일반적 업데이트 규칙 $\theta \leftarrow \theta - \eta \cdot \varphi$ (그래디언트들)” 에서 φ 를 설계하는 문제이며 , Momentum·Adagrad·RMSprop·Adam 은 모두 이 φ 를 점진적으로 개선해온 계보다 .

참고 문헌

References

- Kingma, D. P., & Ba, J. (2015). Adam: A Method for Stochastic Optimization. ICLR.
- Duchi, J., Hazan, E., & Singer, Y. (2011). Adaptive Subgradient Methods for Online Learning and Stochastic Optimization. JMLR.
- Zeiler, M. D. (2012). ADADELTA: An Adaptive Learning Rate Method. arXiv:1212.5701.
- Tieleman, T., & Hinton, G. (2012). Lecture 6.5-RMSprop, COURSERA: Neural Networks for Machine Learning.
- Sutskever, I., Martens, J., Dahl, G., & Hinton, G. (2013). On the Importance of Initialization and Momentum in Deep Learning. ICML.
- Dozat, T. (2016). Incorporating Nesterov Momentum into Adam. ICLR Workshop.
- Loshchilov, I., & Hutter, F. (2019). Decoupled Weight Decay Regularization (AdamW). ICLR.
- Keras 공식 문서 : keras.io/api/optimizers — 옵티마이저 클래스 및 기본값 (2026 년 기준)



감사합니다

Thank You · Questions & Discussion